C++ Linear Data Architecture Subsystem

System Refactoring, Code Audit & Algorithmic Complexity Technical Report

---

Part D: Before-and-After Refactoring Documentation

1. Feature: Dynamic Memory Array Resizing

- **BEFORE:**int* temp = new int[maxCapacity * 2];

  for (int i = 0; i < maxCapacity; i++) temp[i] = dataArray[i];

  dataArray = temp; // Memory Leak!

- **PROBLEM IDENTIFIED:** Old memory buffer allocated to `dataArray` was never deallocated using `delete[]`, resulting in a heap memory leak upon every capacity double.
- **AFTER:**int* temp = new int[newCapacity];

  for (size_t i = 0; i < currentCount; ++i) temp[i] = dataArray[i];

  delete[] dataArray; // Release old buffer

  dataArray = temp;

  maxCapacity = newCapacity;

- **JUSTIFICATION:** Prevents heap memory leaks and safely transfers ownership of dynamically allocated memory buffers.

---

2. Feature: Array Deletion Shifting Logic

- **BEFORE:**for (int i = idx; i < currentCount - 1; i++) {

  for (int j = i; j < currentCount - 1; j++) {

dataArray[j] = dataArray[j + 1]; break;

}

}

- **PROBLEM IDENTIFIED:** Nested loop structure was redundant, inefficient, and unreadable.
- **AFTER:**for (size_t i = static_cast<size_t>(idx); i < currentCount - 1; ++i) {

dataArray[i] = dataArray[i + 1];

}

--currentCount;

- **JUSTIFICATION:** Eliminates unnecessary inner loop overhead and simplifies element shifting to a clear O(n) single-pass routine.

---

3. Feature: Bounds Enforcement & Encapsulation

- **BEFORE:** Global mutable variables (`dataArray`, `currentCount`), missing upper boundary check (`idx >= currentCount`), and off-by-one traversal (`i <= currentCount`).
- **PROBLEM IDENTIFIED:** Destroyed modularity, violated encapsulation, and allowed out-of-bounds reads/writes.
- **AFTER:** Encapsulated inside `DynamicArray` class; enforced `0 <= idx < currentCount` and strict `<` bounds checking.
- **JUSTIFICATION:** Enforces RAII dynamic memory safety, isolates data structures, and eliminates invalid memory access.

Part A: Peer Code Audit Checklist Table

Audit Area	Findings & Lines	Severity	Architectural Impact
Global State	Mutable global variables (`dataArray`, `currentCount`, `maxCapacity`) at lines 8–10	High	Destroys modularity; prevents multiple array instances; thread-unsafe.

Audit Area	Findings & Lines	Severity	Architectural Impact
Memory Leak (Resize)	`temp` allocation without `delete[] dataArray` at lines 18–22	Critical	Heap memory leak on every capacity double; leads to RAM exhaustion.
Memory Leak (Termination)	Program exits without executing `delete[] dataArray` at line 96	High	Heap memory remains unreleased upon execution completion.
Out-of-Bounds (Traversal)	Loop condition `i <= currentCount` at line 58	High	Off-by-one buffer over-read; yields corrupted output or crash.
Out-of-Bounds (Deletion)	`remove_item_at()` lacks upper bound check (`idx >= currentCount`) at line 31	Critical	Attempting deletion at invalid indices causes array corruption or crash.
Inefficient Shift	Redundant inner loop in deletion routine at lines 37–41	Medium	Artificially inflates element shift overhead needlessly.
Naming Standards	Inconsistent naming (`add_item` vs `findItem` vs `printAll`)	Low	Violates unified C++ naming conventions.
Hardcoded Sizing	Fixed matrix sizes (3×3) in `processMatrix()` at lines 66–68	Medium	Prevents dynamic processing of multi-dimensional grids.

Part C: Big-O Algorithmic Complexity Analysis

Subsystem Routine	Best Case	Average Case	Worst Case	Space Complexity	Analysis & Justification
Unamortized Insert (`add_item`)	O(1)	O(1)	O(n)	O(n)	O(1) insertion when capacity remains. Worst case O(n) occurs when array reaches capacity, requiring dynamic reallocation and copying n elements to a new heap buffer.
Linear Deletion (`remove_item_at`)	O(1)	O(n)	O(n)	O(1)	Deleting the last element takes O(1). Deleting at index 0 requires

Subsystem Routine	Best Case	Average Case	Worst Case	Space Complexity	Analysis & Justification
					shifting n-1 elements left. The legacy double loop adds constant-factor overhead without changing asymptotic bounds.
Linear Search (`findItem`)	O(1)	O(n)	O(n)	O(1)	Best case O(1) occurs if target is at index 0. Worst case O(n) occurs if target is at index n-1 or absent.
Array Display (`printAll`)	O(n)	O(n)	O(n)	O(1)	Sequentially visits and displays every element from index 0 to n-1.
Matrix Transpose (`processMatrix`)	O(R x C)	O(R x C)	O(R x C)	O(R x C)	Requires visiting every cell in an R x C grid via nested iteration loops.

Part F: Final Testing & Regression Comparison Table

Test Case ID & Focus	Outcome Before Refactoring	Outcome After Refactoring	Key Verification Observation
TC-01 (Basic Element Insert)	Functional (Global scope)	Functional (Encapsulated)	Data structures safely isolated inside managed class.
TC-02 (Dynamic Memory Resize)	Memory Leak (Heap orphaned)	Zero Leaks (Memory freed)	`delete[]` frees legacy buffers; heap memory remains stable.
TC-03 (Traversal Output)	Off-by-one heap over-read	Clean traversal (0 ... n-1)	Array bounds checked; garbage values eliminated.
TC-06 (Element Deletion)	Shifted with redundant loop	Single-pass O(n) shift	Code execution streamlined; eliminates unnecessary loop conditions.
TC-08 (Upper Boundary Check)	System Crash / Invalid write	Rejects index safely	Enforces bounds checking (`idx >= size`); outputs error safely.

Test Case ID & Focus	Outcome Before Refactoring	Outcome After Refactoring	Key Verification Observation
TC-10 (Program Cleanup)	Leaked dynamic memory	100% Heap Released	Destructor frees heap allocation on program exit.

Guide & Reflection Answers

Section 13: Guide Questions

1. **Data Structure Selection & Trade-Offs**: Dynamic arrays offer contiguous memory storage and O(1) indexed random access, making them preferred for unpredictable workloads. However, dynamic reallocations incur dynamic memory allocation overhead and O(n) element copying costs when resizing occurs.
2. **Memory Hygiene & Lifecycle**: C++ destructors (`~DynamicArray()`) ensure RAII compliance by automatically releasing heap memory via `delete[]`. Omitting `delete[]` causes progressive heap leaks, leading to RAM exhaustion and operating system termination (OOM crashes) in long-running software.
3. **Defensive Programming & Bounds Enforcement**: Raw array indexing (`array[i]`) performs direct pointer arithmetic without checking array boundaries. Omitting upper-bound validation (`idx >= size`) allows buffer overflow/under-read vulnerabilities, enabling attackers to read sensitive memory locations or overwrite process stack/heap frames.
4. **Algorithmic Overhead & Code Smells**: Eliminating the redundant nested loop inside `remove_item_at()` removed unnecessary constant-factor iteration overhead, reducing code complexity to a clean, single-pass O(n) element shift routine.
5. **Value of Peer Code Audits**: Peer code reviews provide structured static code auditing, identifying architectural anti-patterns, memory vulnerabilities, and style inconsistencies prior to production merges.

Section 14: Reflection Questions

1. **Experiential Reflection**: Pinpointing the off-by-one buffer read in `printAll()` (`i <= currentCount`) was particularly tricky because uninitialized heap memory sometimes returned valid-looking values. Systematic unit testing isolated boundary conditions precisely.
2. **Architectural Evaluation**: For systems requiring frequent mid-list insertions or deletions without buffer reallocations, a doubly linked list or queue/deque architecture provides superior performance by avoiding element shifting overhead.

3. **Professional Growth**: Auditing flawed legacy code highlighted the vital importance of RAII memory management, strict input validation, and clean naming standards when engineering resilient C++ systems.